Diplomado en Business Intelligence e Inteligencia Artificial

Módulo 5 · MLOps

Ecuador Growth ML

Nowcast del crecimiento del PIB de Ecuador con Random Forest y despliegue MLOps en Streamlit

Documentación de la ejecución del modelo y de la aplicación

Elaborado por: Victor Oña

Repositorio: https://github.com/VictorHugo1320/Ecuador_Growth_ML_APP

Aplicación: https://appmlopsreentrenamientopy-aah9dtpk6lpswpjr3vp3vj.streamlit.app/

Noviembre de 2026

Proyecto académico con datos simulados. Las estimaciones no son pronósticos oficiales.

1. Objetivo

Construir, ejecutar y documentar un sistema de nowcasting que estima el crecimiento interanual del PIB de Ecuador del trimestre en curso a partir de indicadores económicos de alta frecuencia, aplicando un flujo MLOps completo: entrenamiento y selección de modelos en Google Colab, versionamiento en GitHub y una aplicación web en Streamlit que permite predecir, monitorear y reentrenar el modelo.

El documento presenta: (i) la justificación de las variables elegidas para el Nowcast, (ii) las capturas del código del notebook con sus resultados, y (iii) la aplicación funcionando en cada una de sus pestañas.

2. Arquitectura de la solución

El flujo de trabajo sigue el ciclo de vida MLOps en cuatro etapas:

1. **Datos:** base histórica trimestral 2005Q1–2026Q3 con 20 indicadores candidatos y la variable objetivo `crecimiento_pib`.
2. **Entrenamiento (Colab):** el notebook `Modelo_Nowcast_Dinamico_Ejecutado.ipynb` selecciona variables, valida con series de tiempo, compara 8 configuraciones (experiment tracker) y exporta los artefactos.
3. **Versionamiento (GitHub):** el código, los datos y los artefactos se suben al repositorio `Ecuador_Growth_ML_APP`.
4. **Despliegue (Streamlit):** `app_MLOps_reentrenamiento.py` carga el modelo y su configuración, y ofrece predicción manual, por CSV, seguimiento de experimentos, monitoreo y reentrenamiento supervisado.

Tabla 1. *Artefactos que conectan el notebook con la aplicación.*

Artefacto	Generado por	Uso en la aplicación
`modelo_pib.joblib`	Notebook (celda de exportación)	Pipeline entrenado (imputación + Random Forest) que genera las predicciones
`salidas_tracker/config_modelo.json`	Notebook	Lista de variables y objetivo; la app construye el formulario dinámicamente
`salidas_tracker/experimentos.csv`	Experiment tracker	Grilla de 8 ejecuciones y sus métricas de validación cruzada
`salidas_tracker/experimentos_folds.csv`	Experiment tracker	Errores por cada ventana temporal de validación
`salidas_tracker/mejor_experimento.json`	Experiment tracker	Run ganador, hiperparámetros y métricas de prueba
`salidas_tracker/predicciones_test_ganador.csv`	Evaluación final	Observado vs. predicho 2025Q1–2026Q3 (pestaña Monitoreo)
`requirements.txt`	Configuración	Versiones de librerías para Streamlit Community Cloud

3. Datos utilizados

La base `Base_Historica_PIB_Ecuador_Unidad5.csv` contiene **87 trimestres (2005Q1–2026Q3)** y 22 columnas: el identificador `trimestre`, **20 predictores candidatos** agrupados en siete bloques (actividad económica, sistema financiero, sector externo, sector fiscal, consumo y hogares, mercado laboral, inversión y sector público) y la variable objetivo `crecimiento_pib`. Solo tres variables tienen valores faltantes (`depositos` 2, `confianza_consumidor` 3, `inversion_publica` 2), que el pipeline imputa con la mediana.

Tabla 2. *Partición temporal de la base histórica.*

Partición	Período	Trimestres	Uso
Entrenamiento	2005Q1 – 2024Q4	80	Selección de variables, validación cruzada y tracker
Prueba	2025Q1 – 2026Q3	7	Evaluación independiente, una sola vez
Nowcast	2026Q4	1	Trimestre a estimar (PIB aún no observado)

4. Justificación de la elección de las variables para el Nowcast

Un nowcast busca estimar el PIB del trimestre en curso antes de que el Banco Central del Ecuador publique las Cuentas Nacionales Trimestrales, que llegan con un rezago aproximado de un trimestre. Por ello, los predictores no se eligen solo por su correlación, sino con tres criterios:

1. **Disponibilidad oportuna:** indicadores de alta frecuencia (diarios o mensuales) que se conocen en la fecha de corte.
2. **Relevancia económica** para una economía dolarizada y dependiente del petróleo como la ecuatoriana.
3. **Evidencia empírica:** relación con el PIB y aporte real al error de pronóstico fuera de muestra, medido con validación cruzada temporal usando solo el período de entrenamiento (sin tocar 2025Q1–2026Q3).

4.1 Variables seleccionadas

Tabla 3. *Variables elegidas: un indicador por cada canal clave de la economía.*

Variable	Canal	Justificación económica y de oportunidad
`precio_petroleo`	Sector externo / fiscal	El petróleo es el principal producto de exportación y una fuente central de ingresos fiscales. En dolarización, su precio determina la entrada de divisas, la liquidez interna y la capacidad de gasto público. Se observa a diario.
`credito_privado`	Sistema financiero	El crédito financia el consumo de los hogares y la inversión empresarial, por lo que anticipa la demanda agregada. En dolarización no hay política monetaria propia: el crédito es el principal transmisor del ciclo. Publicación mensual.
`ventas_internas`	Actividad / demanda	Medida directa de la actividad comercial en el mercado interno (declaraciones tributarias). Es un reflejo casi contemporáneo del PIB por el lado de la demanda. Mensual.
`consumo_electrico`	Actividad real	Indicador físico, no monetario, de la producción industrial, comercial y residencial. No depende de precios y está disponible casi en tiempo real.

Con esta combinación, el modelo recoge los cuatro motores del ciclo económico ecuatoriano: el **choque externo** (petróleo), el **financiamiento** (crédito), la **demanda interna** (ventas) y la **producción física** (electricidad). Se evitó incluir varios indicadores del mismo bloque, que aportan información redundante.

4.2 Evidencia: correlación con el crecimiento del PIB

Las cuatro variables elegidas tienen una correlación positiva y relevante con el PIB en el período de entrenamiento: `credito_privado` 0,68; `ventas_internas` 0,63; `consumo_electrico` 0,60 y `precio_petroleo` 0,53. Todas están en la mitad superior de las 20 candidatas.

In [4]:
```
import matplotlib.pyplot as plt

elegidas_estudiante = ["precio_petroleo", "credito_privado", "ventas_internas", "consumo_electrico"]
orden = correlaciones.index[::-1]
colores = ["#1f77b4" if v in elegidas_estudiante else "#c7c7c7" for v in orden]

plt.figure(figsize=(9, 7))
plt.barh(orden, correlaciones.loc[orden, "correlacion_con_PIB"], color=colores)
plt.axvline(0, color="gray", linewidth=0.8)
plt.title("Correlación de cada indicador con el crecimiento del PIB (2005Q1-2024Q4)\nAzul = variables elegidas para el Nowcast")
plt.xlabel("Coeficiente de correlación de Pearson")
plt.grid(axis="x", alpha=0.3)
plt.tight_layout()
plt.show()
```

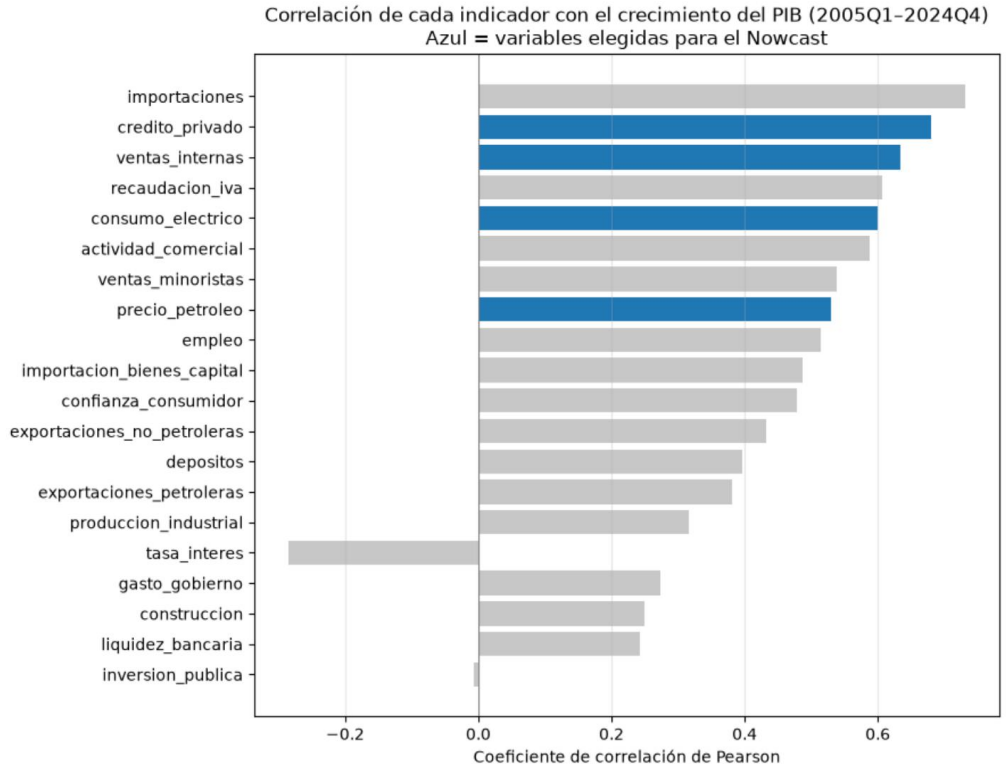


Figura 1. *Correlación de cada candidato con el crecimiento del PIB (azul = variables elegidas). Código y salida del notebook.*

4.3 Evidencia: importancia en un Random Forest con las 20 candidatas

Al entrenar un Random Forest con las 20 candidatas, `credito_privado` (2.º), `consumo_electrico` (3.º) y `precio_petroleo` (4.º) aparecen entre las cuatro variables más importantes, tanto por impureza como por permutación. `ventas_internas` queda más abajo (9.º) porque comparte

información con el crédito y la electricidad, y el bosque reparte la importancia entre variables correlacionadas.

In [5]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.inspection import permutation_importance

# Random Forest con las 20 candidatas: ¿cuáles usa más el modelo?
rf_todas = Pipeline([
    ("imputar", SimpleImputer(strategy="median")),
    ("modelo", RandomForestRegressor(n_estimators=300, max_depth=5, min_samples_leaf=3,
                                     random_state=2026, n_jobs=-1)),
])
rf_todas.fit(hist_train[candidatas], hist_train["crecimiento_pib"])

importancias = pd.DataFrame({
    "importancia_impureza": rf_todas.named_steps["modelo"].feature_importances_,
}, index=candidatas)

perm = permutation_importance(rf_todas, hist_train[candidatas], hist_train["crecimiento_pib"],
                              n_repeats=20, random_state=2026, scoring="neg_root_mean_squared_error")
importancias["importancia_permutacion"] = perm.importances_mean
importancias = importancias.sort_values("importancia_impureza", ascending=False)
importancias["elegida"] = importancias.index.isin(elegidas_estudiante)
display(importancias.round(4))

print("Participación de las 4 variables elegidas en la importancia total:",
      round(importancias.loc[elegidas_estudiante, "importancia_impureza"].sum() * 100, 1), "%")
```

	importancia_impureza	importancia_permutacion	elegida
importaciones	0.3561	0.6213	False
credito_privado	0.1694	0.3719	True
consumo_electrico	0.0986	0.1686	True
precio_petroleo	0.0656	0.1256	True
actividad_comercial	0.0393	0.0608	False
exportaciones_no_petroleras	0.0381	0.0777	False
produccion_industrial	0.0352	0.0901	False
recaudacion_iva	0.0339	0.0403	False
ventas_internas	0.0279	0.0350	True
liquidez_bancaria	0.0247	0.0443	False
tasa_interes	0.0232	0.0453	False
depositos	0.0192	0.0306	False
importacion_bienes_capital	0.0161	0.0254	False
empleo	0.0109	0.0178	False
gasto_gobierno	0.0099	0.0141	False
construccion	0.0080	0.0132	False
ventas_minoristas	0.0080	0.0090	False
confianza_consumidor	0.0060	0.0056	False
inversion_publica	0.0058	0.0114	False
exportaciones_petroleras	0.0041	0.0073	False

```
Participación de las 4 variables elegidas en la importancia total: 36.1 %
```

Figura 2. *Importancia por impureza y por permutación con las 20 candidatas.*

4.4 Evidencia decisiva: error de pronóstico fuera de muestra

La prueba más importante para un nowcast es cuánto se equivoca el modelo al predecir trimestres que no ha visto. Se compararon diez conjuntos de predictores con la misma validación cruzada temporal (4 ventanas de 4 trimestres, 2021–2024) y el mismo Random Forest:

Tabla 4. *Comparación de conjuntos de variables por validación cruzada temporal (menor es mejor).*

Conjunto de predictores	N.º var.	MAE CV	RMSE CV
Elegidas (petróleo, crédito, ventas, electricidad)	4	0,976	**1,081**
Sin `ventas_internas`	3	0,957	1,082
Sin `consumo_electrico`	3	0,987	1,095
Sin `precio_petroleo`	3	0,999	1,106
Elegidas + `importaciones`	5	1,111	1,240
Las 20 candidatas	20	1,125	1,253
Top-4 por importancia RF	4	1,127	1,263
Top-4 por correlación	4	1,180	1,337
Sin `credito_privado`	3	1,180	1,411
Solo `precio_petroleo`	1	1,997	2,236

Los resultados respaldan la selección:

- El conjunto elegido obtiene el **menor RMSE de todos (1,081)**. Usar las 20 variables empeora el error en 16 % (1,253): con solo 80 trimestres de entrenamiento, las variables adicionales introducen ruido y sobreajuste.
- Elegir las variables solo por correlación también es peor (1,337): la correlación no garantiza poder predictivo fuera de muestra.
- **El crédito privado es el predictor decisivo:** sin él, el RMSE sube de 1,08 a 1,41. Sin `precio_petroleo` sube a 1,11 y sin `consumo_electrico` a 1,10.
- **¿Por qué no se incluyeron las importaciones?** Son la variable de mayor correlación (0,73), pero al agregarlas el error empeora (1,240). Económicamente, las importaciones son más una consecuencia del nivel de actividad —suben cuando crece el ingreso— que un indicador adelantado, y su relación con el PIB es menos estable en el tiempo.
- **Las ventas internas son el caso más débil:** su aporte marginal es casi nulo (sin ella el RMSE queda en 1,082 y el MAE baja levemente). Se mantiene porque es la medida más directa de la demanda interna y da robustez si otro indicador llega tarde o con errores, pero es la primera candidata a revisarse en una próxima versión.

In [6]:

```python
from sklearn.model_selection import TimeSeriesSplit, cross_validate

# Comparamos conjuntos alternativos de predictores con la MISMA validación temporal
cv_sel = TimeSeriesSplit(n_splits=4, test_size=4)
top_corr_4 = correlaciones.index[:4].tolist()
top_imp_4 = importancias.index[:4].tolist()

conjuntos = {
    "Elegidas (petróleo, crédito, ventas, electricidad)": elegidas_estudiante,
    "Top-4 por correlación": top_corr_4,
    "Top-4 por importancia RF": top_imp_4,
    "Las 20 candidatas": candidatas,
    "Solo precio_petroleo": ["precio_petroleo"],
    "Elegidas + importaciones": elegidas_estudiante + ["importaciones"],
}
# Prueba de exclusión: quitamos una variable elegida a la vez
for v in elegidas_estudiante:
    conjuntos[f"Sin {v}"] = [w for w in elegidas_estudiante if w != v]

filas = []
for nombre, vars_ in conjuntos.items():
    pipe = Pipeline([
        ("imputar", SimpleImputer(strategy="median")),
        ("modelo", RandomForestRegressor(n_estimators=180, max_depth=5, min_samples_leaf=3,
                                         random_state=2026, n_jobs=-1)),
    ])
    r = cross_validate(pipe, hist_train[vars_], hist_train["crecimiento_pib"], cv=cv_sel,
                       scoring={"MAE": "neg_mean_absolute_error", "RMSE": "neg_root_mean_squared_error"})
    filas.append({"conjunto": nombre, "n_variables": len(vars_), "variables": ", ".join(vars_),
                  "MAE_CV": -r["test_MAE"].mean(), "RMSE_CV": -r["test_RMSE"].mean()})

comparacion_conjuntos = pd.DataFrame(filas).sort_values("RMSE_CV").reset_index(drop=True)
display(comparacion_conjuntos.round(3))
```

	conjunto	n_variables	variables	MAE_CV	RMSE_CV
0	Elegidas (petróleo, crédito, ventas, electrici...	4	precio_petroleo, credito_privado, ventas_inter...	0.976	1.081
1	Sin ventas_internas	3	precio_petroleo, credito_privado, consumo_elec...	0.957	1.082
2	Sin consumo_electrico	3	precio_petroleo, credito_privado, ventas_internas	0.987	1.095
3	Sin precio_petroleo	3	credito_privado, ventas_internas, consumo_elec...	0.999	1.106
4	Elegidas + importaciones	5	precio_petroleo, credito_privado, ventas_inter...	1.111	1.240
5	Las 20 candidatas	20	ventas_internas, produccion_industrial, activi...	1.125	1.253
6	Top-4 por importancia RF	4	importaciones, credito_privado, consumo_electr...	1.127	1.263
7	Top-4 por correlación	4	importaciones, credito_privado, ventas_interna...	1.180	1.337
8	Sin credito_privado	3	precio_petroleo, ventas_internas, consumo_elec...	1.180	1.411
9	Solo precio_petroleo	1	precio_petroleo	1.997	2.236

Figura 3. *Código del notebook que compara los conjuntos de variables y su resultado.*

4.5 Conclusión sobre la selección

Se eligieron **cuatro indicadores oportunos y económicamente interpretables**, uno por cada canal del ciclo ecuatoriano. La evidencia muestra que este conjunto pequeño pronostica mejor que alternativas más grandes o elegidas solo por correlación, reduce el riesgo de sobreajuste y hace que la aplicación sea fácil de alimentar: el analista solo necesita cuatro datos para generar el nowcast del trimestre.

5. Desarrollo del modelo en Google Colab: código y ejecución

A continuación se muestran las capturas del notebook ejecutado de principio a fin, con el código y la salida de cada celda. El archivo `Modelo_Nowcast_Dinamico_Ejecutado.ipynb` contiene todas las ejecuciones.

5.1 Carga de la base histórica y del diccionario de datos

In [1]:
```
import os
import pandas as pd

# En Colab los archivos se suben a /content; en ejecución local se leen de la carpeta del notebook.
RUTA_DATOS = "/content/" if os.path.isdir("/content") else ""

historico = pd.read_csv(RUTA_DATOS + "Base_Historica_PIB_Ecuador_Unidad5.csv")

print(historico.shape)
historico.head()
```

```
(87, 22)
```

Out[1]:

	trimestre	ventas_internas	produccion_industrial	actividad_comercial	consumo_electrico	credito_privado	depositos	tasa_interes	liquidez_bancaria
0	2005Q1	3.256	2.999	2.271	2.679	6.302	4.049	8.918	106.381
1	2005Q2	1.289	3.505	2.059	2.559	4.723	4.312	9.149	104.094
2	2005Q3	2.669	2.851	2.608	3.164	7.036	4.998	8.916	109.964
3	2005Q4	3.668	1.757	2.939	2.359	7.695	3.973	9.044	113.708
4	2006Q1	2.699	4.183	3.466	3.540	7.717	5.231	7.781	109.912

5 rows × 22 columns

Figura 4. *Carga de la base histórica: 87 trimestres y 22 columnas.*

In [2]:

```
diccionario = pd.read_csv(
    RUTA_DATOS + "Diccionario_Datos_PIB_Ecuador_Unidad5.csv"
)

# Revisar las variables y sus definiciones
display(diccionario)
```

	variable	rol	grupo	unidad	interpretacion_didactica	descripcion	valores_faltantes_historico
0	trimestre	Tiempo	Identificador temporal	trimestre calendario	Trimestre al que corresponde cada observación.	Período de referencia	0
1	ventas_internas	Predictor candidato (X)	Actividad económica	variación porcentual simulada (%)	Variación simulada de las ventas realizadas en...	Indicador ficticio supuesto disponible a la fe...	0
2	produccion_industrial	Predictor candidato (X)	Actividad económica	variación porcentual simulada (%)	Variación simulada de la actividad de producci...	Indicador ficticio supuesto disponible a la fe...	0
3	actividad_comercial	Predictor candidato (X)	Actividad económica	variación porcentual simulada (%)	Variación simulada del nivel de actividad come...	Indicador ficticio supuesto disponible a la fe...	0
4	consumo_electrico	Predictor candidato (X)	Actividad económica	variación porcentual simulada (%)	Variación simulada del consumo de energía eléc...	Indicador ficticio supuesto disponible a la fe...	0
5	credito_privado	Predictor candidato (X)	Sistema financiero	variación porcentual simulada (%)	Variación simulada del crédito otorgado al sec...	Indicador ficticio supuesto disponible a la fe...	0
6	depositos	Predictor candidato (X)	Sistema financiero	variación porcentual simulada (%)	Variación simulada de los depósitos del sistem...	Indicador ficticio supuesto disponible a la fe...	2
7	tasa_interes	Predictor candidato (X)	Sistema financiero	porcentaje simulado	Tasa de interés expresada en porcentaje en la ...	Indicador ficticio supuesto disponible a la fe...	0
8	liquidez_bancaria	Predictor candidato (X)	Sistema financiero	índice simulado (puntos)	Índice ficticio de liquidez del sistema bancario.	Indicador ficticio supuesto disponible a la fe...	0
9	exportaciones_petroleras	Predictor candidato (X)	Sector externo	variación porcentual simulada (%)	Variación simulada de las exportaciones petrol...	Indicador ficticio supuesto disponible a la fe...	0
10	exportaciones_no_petroleras	Predictor candidato (X)	Sector externo	variación porcentual simulada (%)	Variación simulada de las exportaciones no pet...	Indicador ficticio supuesto disponible a la fe...	0
11	importaciones	Predictor candidato (X)	Sector externo	variación porcentual simulada (%)	Variación simulada del valor o volumen agregad...	Indicador ficticio supuesto disponible a la fe...	0
12	precio_petroleo	Predictor candidato (X)	Sector externo	USD por barril ficticios	Precio simulado del petróleo, en dólares por b...	Indicador ficticio supuesto disponible a la fe...	0
13	recaudacion_iva	Predictor candidato (X)	Sector fiscal	variación porcentual simulada (%)	Variación simulada de la recaudación por IVA.	Indicador ficticio supuesto disponible a la fe...	0
14	confianza_consumidor	Predictor candidato (X)	Consumo y hogares	índice simulado (puntos)	Índice simulado de confianza del consumidor.	Indicador ficticio supuesto disponible a la fe...	3
15	ventas_minoristas	Predictor candidato (X)	Consumo y hogares	variación porcentual simulada (%)	Variación simulada de ventas minoristas.	Indicador ficticio supuesto disponible a la fe...	0
16	empleo	Predictor candidato (X)	Mercado laboral	porcentaje simulado	Indicador simulado de empleo expresado en porc...	Indicador ficticio supuesto disponible a la fe...	0
17	inversion_publica	Predictor candidato (X)	Inversión y sector público	variación porcentual simulada (%)	Variación simulada de la inversión pública.	Indicador ficticio supuesto disponible a la fe...	2
18	gasto_gobierno	Predictor candidato (X)	Inversión y sector público	variación porcentual simulada (%)	Variación simulada del gasto del gobierno.	Indicador ficticio supuesto disponible a la fe...	0
19	construccion	Predictor candidato (X)	Inversión y sector público	variación porcentual simulada (%)	Variación simulada de la actividad de construc...	Indicador ficticio supuesto disponible a la fe...	0
20	importacion_bienes_capital	Predictor candidato (X)	Inversión y sector público	variación porcentual simulada (%)	Variación simulada de las importaciones de bie...	Indicador ficticio supuesto disponible a la fe...	0
21	crecimiento_pib	Objetivo (Y)	Variable objetivo	variación interanual simulada (%)	Crecimiento interanual simulado del PIB; varia...	Objetivo del modelo; Q4 2026 sin valor observado	0

Figura 5. *Diccionario de datos con el rol, grupo y unidad de cada variable. (2 partes)*

5.2 Selección de las variables predictoras

In [7]:
```
# Variable objetivo
TARGET = "crecimiento_pib"

# PERSONALIZACIÓN DEL ESTUDIANTE: cambiar SOLO esta lista.
# Elegir entre 4 y 8 indicadores numéricos existentes en el CSV.
FEATURES = [
    "precio_petroleo",
    "credito_privado",
    "ventas_internas",
    "consumo_electrico"
]

if not 4 <= len(FEATURES) <= 8:
    raise ValueError("Seleccione entre 4 y 8 predictores")
if len(FEATURES) != len(set(FEATURES)):
    raise ValueError("No repita predictores")
faltantes = [v for v in FEATURES + [TARGET, "trimestre"] if v not in historico.columns]
if faltantes:
    raise ValueError(f"Columnas inexistentes: {faltantes}")
no_numericas = [v for v in FEATURES if not pd.api.types.is_numeric_dtype(historico[v])]
if no_numericas:
    raise ValueError(f"Estas variables no son numéricas: {no_numericas}")

print("Variable objetivo:", TARGET)
print("Predictores seleccionados:", FEATURES)
```

```
Variable objetivo: crecimiento_pib
Predictores seleccionados: ['precio_petroleo', 'credito_privado', 'ventas_internas', 'consumo_electrico']
```

Figura 6. *Definición de TARGET y FEATURES, con validaciones (4 a 8 variables, sin duplicados, numéricas).*

In [8]:
```
# X contiene las variables predictoras
X = historico[FEATURES]

# y contiene la variable objetivo
y = historico[TARGET]

# Verificamos las dimensiones
print("Dimensiones de X:", X.shape)
print("Dimensiones de y:", y.shape)

# Visualizamos los predictores
display(X.head())
```

```
Dimensiones de X: (87, 4)
Dimensiones de y: (87,)
```

	precio_petroleo	credito_privado	ventas_internas	consumo_electrico
0	67.379	6.302	3.256	2.679
1	64.620	4.723	1.289	2.559
2	74.172	7.036	2.669	3.164
3	66.515	7.695	3.668	2.359
4	65.541	7.717	2.699	3.540

Figura 7. *Construcción de la matriz X y del vector y.*

5.3 Partición temporal entrenamiento / prueba

La partición respeta el orden del tiempo: el modelo nunca se entrena con datos posteriores a los que debe predecir.

In [9]:
```python
# Ordenamos la base cronológicamente
historico = historico.sort_values("trimestre").reset_index(drop=True)

# Volvemos a crear X e y después de ordenar los trimestres
X = historico[FEATURES]
y = historico[TARGET]

# Entrenamiento: 2005Q1 hasta 2024Q4
train = historico[
    historico["trimestre"] <= "2024Q4"
].copy()

# Prueba: 2025Q1 hasta 2026Q3
test = historico[
    historico["trimestre"] >= "2025Q1"
].copy()

# Separamos las variables
X_train = train[FEATURES]
y_train = train[TARGET]

X_test = test[FEATURES]
y_test = test[TARGET]

# Comprobamos las particiones
print("Entrenamiento:", len(train))
print("Prueba:", len(test))

print("Primero en prueba:", test["trimestre"].iloc[0])
print("Último en prueba:", test["trimestre"].iloc[-1])
```

```
Entrenamiento: 80
Prueba: 7
Primero en prueba: 2025Q1
Último en prueba: 2026Q3
```

Figura 8. *Separación cronológica: 80 trimestres de entrenamiento y 7 de prueba.*

5.4 Preprocesamiento y pipeline

Encapsular la imputación dentro del pipeline evita fuga de información y garantiza que la app aplique exactamente el mismo tratamiento a los datos nuevos.

In [10]:
```python
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer

# Definimos la preparación de variables
preprocesador = ColumnTransformer(
    transformers=[
        (
            "numericas",
            SimpleImputer(strategy="median"),
            FEATURES
        )
    ]
)
```

Figura 9. *Imputación por mediana dentro de un ColumnTransformer.*

In [11]:
```python
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor

pipeline_rf = Pipeline(steps=[
    (
        "preprocesamiento",
        preprocesador
    ),
    (
        "modelo",
        RandomForestRegressor(
            n_estimators=180,
            max_depth=5,
            min_samples_leaf=3,
            max_features=1.0,
            random_state=2026,
            n_jobs=-1
        )
    )
])

print(pipeline_rf)
```

```
Pipeline(steps=[('preprocesamiento',
                 ColumnTransformer(transformers=[('numericas',
                                                  SimpleImputer(strategy='median'),
                                                  ['precio_petroleo',
                                                   'credito_privado',
                                                   'ventas_internas',
                                                   'consumo_electrico'])])),
                ('modelo',
                 RandomForestRegressor(max_depth=5, min_samples_leaf=3,
                                       n_estimators=180, n_jobs=-1,
                                       random_state=2026))])
```

Figura 10. *Pipeline: preprocesamiento + RandomForestRegressor (random_state = 2026).*

5.5 Validación cruzada temporal

In [13]:
```python
from sklearn.model_selection import TimeSeriesSplit, cross_validate

# Definimos la validación cruzada temporal
validacion = TimeSeriesSplit(
    n_splits=4,
    test_size=4
)

# Revisamos los períodos de cada partición
for numero, (indices_train, indices_valid) in enumerate(
    validacion.split(X_train), start=1
):
    print("Validación:", numero)

    print(
        "Entrenamiento:",
        train.iloc[indices_train]["trimestre"].iloc[0],
        "hasta",
        train.iloc[indices_train]["trimestre"].iloc[-1]
    )

    print(
        "Validación:",
        train.iloc[indices_valid]["trimestre"].iloc[0],
        "hasta",
        train.iloc[indices_valid]["trimestre"].iloc[-1]
    )

    print("-------------------------")
```

```
Validación: 1
Entrenamiento: 2005Q1 hasta 2020Q4
Validación: 2021Q1 hasta 2021Q4
-------------------------
Validación: 2
Entrenamiento: 2005Q1 hasta 2021Q4
Validación: 2022Q1 hasta 2022Q4
-------------------------
Validación: 3
Entrenamiento: 2005Q1 hasta 2022Q4
Validación: 2023Q1 hasta 2023Q4
-------------------------
Validación: 4
Entrenamiento: 2005Q1 hasta 2023Q4
Validación: 2024Q1 hasta 2024Q4
-------------------------
```

Figura 11. *Ventanas de TimeSeriesSplit: 4 particiones de 4 trimestres (2021–2024).*

In [14]:
```python
# Definimos las métricas de evaluación
metricas = {
    "MAE": "neg_mean_absolute_error",
    "RMSE": "neg_root_mean_squared_error"
}

# Ejecutamos la validación cruzada
resultados_cv = cross_validate(
    pipeline_rf,
    X_train,
    y_train,
    cv=validacion,
    scoring=metricas
)

# Convertimos los resultados en una tabla
tabla_cv = pd.DataFrame({
    "Validacion": [1, 2, 3, 4],
    "MAE": -resultados_cv["test_MAE"],
    "RMSE": -resultados_cv["test_RMSE"]
})

display(tabla_cv)
```

	Validacion	MAE	RMSE
0	1	0.851095	0.884332
1	2	1.298294	1.403993
2	3	0.849781	0.880243
3	4	0.903296	1.155092

Figura 12. *MAE y RMSE por ventana de validación.*

In [15]:
```python
print("MAE promedio:", round(tabla_cv["MAE"].mean(), 3))

print("RMSE promedio:", round(tabla_cv["RMSE"].mean(), 3))
```

```
MAE promedio: 0.976
RMSE promedio: 1.081
```

Figura 13. *Error promedio de la validación cruzada del modelo base.*

5.6 Evaluación en el período de prueba

In [16]:
```python
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error,
    root_mean_squared_error, r2_score
)

# Entrenamiento con 2005Q1–2024Q4
pipeline_rf.fit(X_train, y_train)

# Predicción de los 7 trimestres reservados
predicciones_test = pipeline_rf.predict(X_test)

# Métricas de prueba
mae_test = mean_absolute_error(y_test, predicciones_test)
mse_test = mean_squared_error(y_test, predicciones_test)
rmse_test = root_mean_squared_error(y_test, predicciones_test)
r2_test = r2_score(y_test, predicciones_test)

print("MAE prueba:", round(mae_test, 3))
print("MSE prueba:", round(mse_test, 3))
print("RMSE prueba:", round(rmse_test, 3))
print("R² prueba:", round(r2_test, 3))

# Creamos el DataFrame de resultados para mostrar
resultados_test = pd.DataFrame({
    "Trimestre": test["trimestre"],
    "PIB_real": y_test,
    "PIB_predicho": predicciones_test,
    "Residuo": y_test - predicciones_test
})

display(resultados_test)
```

```
MAE prueba: 0.716
MSE prueba: 0.915
RMSE prueba: 0.957
R² prueba: 0.431
```

	Trimestre	PIB_real	PIB_predicho	Residuo
80	2025Q1	-2.055	-0.009048	-2.045952
81	2025Q2	-0.617	-0.023996	-0.593004
82	2025Q3	0.768	0.655496	0.112504
83	2025Q4	1.273	0.929258	0.343742
84	2026Q1	-0.659	0.389094	-1.048094
85	2026Q2	-0.800	-0.002602	-0.797398
86	2026Q3	1.840	1.908404	-0.068404

Figura 14. *Entrenamiento con 2005Q1–2024Q4 y predicción de 2025Q1–2026Q3.*

In [19]:

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

plt.plot(
    resultados_test["Trimestre"],
    resultados_test["PIB_real"],
    marker="o",
    label="PIB observado"
)

plt.plot(
    resultados_test["Trimestre"],
    resultados_test["PIB_predicho"],
    marker="s",
    label="PIB predicho"
)

plt.axhline(0, color="gray", linestyle="--")

plt.title("Ecuador: crecimiento del PIB observado y predicho")
plt.xlabel("Trimestre")
plt.ylabel("Crecimiento interanual del PIB (%)")

plt.xticks(rotation=45)
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

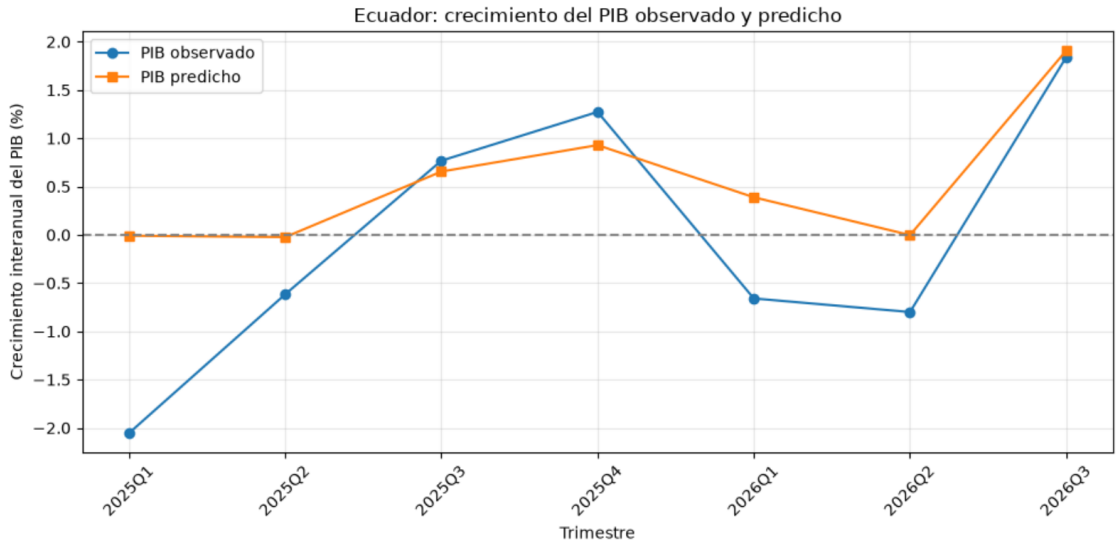


Figura 15. *Crecimiento del PIB observado vs. predicho en el período de prueba.*

5.7 Experiment tracker: 8 configuraciones de hiperparámetros

El ganador se elige únicamente con la validación cruzada del entrenamiento; el período de prueba se usa una sola vez, después de la selección, para no sesgar la evaluación.

In [20]:

```python
from datetime import datetime, timezone
from itertools import product
from sklearn.base import clone
from sklearn.model_selection import cross_validate

# Cuatro particiones temporales de cuatro trimestres, como en el ejercicio original
cv_tracker = TimeSeriesSplit(n_splits=4, test_size=4)

# Cuadrícula pequeña y didáctica: 2 × 2 × 2 = 8 experimentos
combinaciones = list(product([100, 200], [4, 7], [2, 4]))
registros_runs = []
registros_folds = []
fecha_ejecucion = datetime.now(timezone.utc).isoformat(timespec="seconds")

for numero, (arboles, profundidad, hoja) in enumerate(combinaciones, start=1):
    run_id = f"RUN-{numero:03d}"
    candidato = clone(pipeline_rf)
    candidato.set_params(
        modelo__n_estimators=arboles,
        modelo__max_depth=profundidad,
        modelo__min_samples_leaf=hoja
    )
    puntajes = cross_validate(
        candidato, X_train, y_train, cv=cv_tracker,
        scoring={"MAE": "neg_mean_absolute_error", "RMSE": "neg_root_mean_squared_error"},
        error_score="raise"
    )
    maes = -puntajes["test_MAE"]
    rmses = -puntajes["test_RMSE"]
    registros_runs.append({
        "run_id": run_id, "fecha_utc": fecha_ejecucion,
        "modelo": "Random Forest", "n_estimators": arboles,
        "max_depth": profundidad, "min_samples_leaf": hoja,
        "max_features": 1.0, "random_state": 2026,
        "n_splits": 4, "test_size": 4,
        "periodo_entrenamiento_fin": str(train["trimestre"].iloc[-1]),
        "n_variables": len(FEATURES), "variables": ", ".join(FEATURES),
        "mae_cv": float(maes.mean()), "rmse_cv": float(rmses.mean()),
        "std_rmse_cv": float(rmses.std(ddof=1)),
    })
    for fold, (mae, rmse) in enumerate(zip(maes, rmses), start=1):
        indices_train, indices_valid = list(cv_tracker.split(X_train))[fold-1]
        registros_folds.append({
            "run_id": run_id, "fold": fold,
            "fin_entrenamiento": str(train.iloc[indices_train]["trimestre"].iloc[-1]),
            "inicio_validacion": str(train.iloc[indices_valid]["trimestre"].iloc[0]),
            "fin_validacion": str(train.iloc[indices_valid]["trimestre"].iloc[-1]),
            "mae": float(mae), "rmse": float(rmse)
        })

experimentos = pd.DataFrame(registros_runs).sort_values(["rmse_cv", "mae_cv"]).reset_index(drop=True)
experimentos["ranking"] = range(1, len(experimentos)+1)
experimentos["es_mejor"] = experimentos["ranking"] == 1
folds_experimentos = pd.DataFrame(registros_folds)

display(experimentos[["run_id", "n_estimators", "max_depth", "min_samples_leaf", "mae_cv", "rmse_cv", "ranking"]])
```

	run_id	n_estimators	max_depth	min_samples_leaf	mae_cv	rmse_cv	ranking
0	RUN-003	100	7	2	0.923641	1.019223	1
1	RUN-007	200	7	2	0.946759	1.044938	2
2	RUN-001	100	4	2	0.993926	1.098730	3
3	RUN-005	200	4	2	1.003794	1.112439	4
4	RUN-004	100	7	4	1.008598	1.116670	5
5	RUN-008	200	7	4	1.013061	1.124975	6
6	RUN-002	100	4	4	1.028889	1.143337	7
7	RUN-006	200	4	4	1.031227	1.148241	8

Figura 16. *Grilla 2×2×2 (n_estimators, max_depth, min_samples_leaf) evaluada con las mismas ventanas temporales.*

In [21]:

```python
# Seleccionamos el mejor modelo usando ÚNICAMENTE validación cruzada
mejor = experimentos.iloc[0]
mejor_run_id = mejor["run_id"]
modelo_ganador = clone(pipeline_rf).set_params(
    modelo__n_estimators=int(mejor["n_estimators"]),
    modelo__max_depth=int(mejor["max_depth"]),
    modelo__min_samples_leaf=int(mejor["min_samples_leaf"])
)

# Evaluación final del ganador en el período de prueba reservado
modelo_ganador.fit(X_train, y_train)
predicciones_ganador = modelo_ganador.predict(X_test)
metricas_ganador_test = {
    "run_id": mejor_run_id,
    "mae_test": float(mean_absolute_error(y_test, predicciones_ganador)),
    "rmse_test": float(root_mean_squared_error(y_test, predicciones_ganador)),
    "r2_test": float(r2_score(y_test, predicciones_ganador)),
    "n_test": len(X_test),
    "inicio_test": str(test["trimestre"].iloc[0]),
    "fin_test": str(test["trimestre"].iloc[-1]),
}

# No comparamos múltiples modelos con el test para volver a elegir.
print("Run seleccionado por validación cruzada:", mejor_run_id)
print("RMSE CV:", round(float(mejor["rmse_cv"]), 3))
print("RMSE prueba independiente:", round(metricas_ganador_test["rmse_test"], 3))

test_ganador = pd.DataFrame({
    "trimestre": test["trimestre"].to_numpy(),
    "pib_observado": y_test.to_numpy(),
    "pib_predicho": predicciones_ganador,
})
test_ganador["error"] = test_ganador["pib_observado"] - test_ganador["pib_predicho"]
```

```
Run seleccionado por validación cruzada: RUN-003
RMSE CV: 1.019
RMSE prueba independiente: 0.83
```

Figura 17. *Selección del ganador solo con validación cruzada y evaluación única en prueba.*

5.8 Exportación de artefactos y modelo final

In [22]:
```python
# Archivos que alimentarán el nuevo panel de Streamlit
from pathlib import Path
import json

carpeta_salidas = Path("salidas_tracker")
carpeta_salidas.mkdir(exist_ok=True)
experimentos.to_csv(carpeta_salidas / "experimentos.csv", index=False)
folds_experimentos.to_csv(carpeta_salidas / "experimentos_folds.csv", index=False)
test_ganador.to_csv(carpeta_salidas / "predicciones_test_ganador.csv", index=False)
with open(carpeta_salidas / "mejor_experimento.json", "w", encoding="utf-8") as archivo:
    json.dump({
        "run_id": mejor_run_id,
        "parametros": {
            "n_estimators": int(mejor["n_estimators"]),
            "max_depth": int(mejor["max_depth"]),
            "min_samples_leaf": int(mejor["min_samples_leaf"])
        },
        "metricas_cv": {
            "mae": float(mejor["mae_cv"]),
            "rmse": float(mejor["rmse_cv"])
        },
        "metricas_test": metricas_ganador_test,
        "features": FEATURES
    }, archivo, indent=2, ensure_ascii=False)

print("Archivos disponibles:")
for archivo in carpeta_salidas.iterdir():
    print("-", archivo)

# Descargar todos los resultados juntos desde Colab (opcional)
# !zip -r salidas_tracker.zip salidas_tracker
# from google.colab import files
# files.download("salidas_tracker.zip")

# Configuración que Streamlit utilizará para crear formularios dinámicos.
config_modelo = {
    "modelo": "Random Forest",
    "target": TARGET,
    "features": FEATURES,
    "run_id": mejor_run_id,
    "periodicidad": "trimestral",
    "datos": "simulados",
    "version_config": 1
}
with open(carpeta_salidas / "config_modelo.json", "w", encoding="utf-8") as archivo:
    json.dump(config_modelo, archivo, indent=2, ensure_ascii=False)
print("Configuración del modelo exportada:", config_modelo)
```

```
Archivos disponibles:
- salidas_tracker\config_modelo.json
- salidas_tracker\experimentos.csv
- salidas_tracker\experimentos_folds.csv
- salidas_tracker\mejor_experimento.json
- salidas_tracker\predicciones_test_ganador.csv
Configuración del modelo exportada: {'modelo': 'Random Forest', 'target': 'crecimiento_pib', 'features': ['precio_petroleo', 'credito_pri
vado', 'ventas_internas', 'consumo_electrico'], 'run_id': 'RUN-003', 'periodicidad': 'trimestral', 'datos': 'simulados', 'version_confi
g': 1}
```

Figura 18. *Exportación de los CSV y JSON del tracker y de config_modelo.json.*

In [23]:
```python
# Reentrenamos el mejor pipeline con TODA la historia observada
pipeline_rf = clone(modelo_ganador)
pipeline_rf.fit(X, y)

print("Modelo final entrenado correctamente")
print("Run ganador:", mejor_run_id)
print("Observaciones utilizadas:", len(X))
print("Variables utilizadas:", FEATURES)
```

```
Modelo final entrenado correctamente
Run ganador: RUN-003
Observaciones utilizadas: 87
Variables utilizadas: ['precio_petroleo', 'credito_privado', 'ventas_internas', 'consumo_electrico']
```

Figura 19. *Reentrenamiento del ganador con los 87 trimestres observados.*

In [24]:
```python
import joblib

# Guardamos el pipeline completo
joblib.dump(
    pipeline_rf,
    "modelo_pib.joblib"
)

print("Pipeline guardado correctamente")
```

```
Pipeline guardado correctamente
```

Figura 20. *Serialización del pipeline en modelo_pib.joblib.*

```
In [26]: import sys
         import sklearn
         import numpy
         import pandas
         import joblib

         print("Python:", sys.version.split()[0])
         print("scikit-learn:", sklearn.__version__)
         print("numpy:", numpy.__version__)
         print("pandas:", pandas.__version__)
         print("joblib:", joblib.__version__)

Python: 3.12.3
scikit-learn: 1.9.1
numpy: 2.5.4
pandas: 3.1.1
joblib: 1.6.0
```

Figura 21. *Versiones de librerías utilizadas.*

5.9 Resumen de resultados del modelo

Tabla 5. *Métricas del modelo base y del ganador del tracker.*

Modelo	Hiperparámetros	MAE CV	RMSE CV	MAE prueba	RMSE prueba	R^2 prueba
Base	180 árboles, prof. 5, hoja 3	0,976	1,081	0,716	0,957	0,431
RUN-003 (ganador)	100 árboles, prof. 7, hoja 2	0,924	1,019	0,632	0,830	0,572

El ganador reduce el RMSE de prueba de 0,957 a 0,830 puntos porcentuales (−13 %) y explica el 57 % de la variabilidad del PIB en trimestres no vistos. El mayor error se concentra en 2025Q1 (−2,06 % observado frente a −0,36 % predicho): el modelo anticipa la dirección de la caída, pero no su intensidad, algo habitual en los Random Forest, que tienden a suavizar los valores extremos.

6. Aplicación Streamlit

6.1 Código principal de la aplicación

La aplicación `app_MLOps_reentrenamiento.py` no define las variables a mano: las lee de `config_modelo.json` y verifica que coincidan con las del modelo serializado. Así, si se cambia la selección en Colab, la app se adapta sola. Fragmentos clave:

app_MLOps_reentrenamiento.py — línea 99

```
# 3. CARGAR MODELO ENTRENADO
# ============================================================

ruta_modelo = localizar_archivo(
    "modelo_pib.joblib",
    ["modelos", "."]
)

if ruta_modelo is None:
    st.error(
        "No se encontró modelo_pib.joblib. "
        "Colócalo en la carpeta modelos/ "
        "o en la raíz del repositorio."
    )
    st.stop()

try:
    modelo = cargar_modelo(str(ruta_modelo))
except Exception as e:
    st.error(f"No se pudo cargar el modelo: {e}")
    st.stop()
```

Figura 22. *Carga del modelo modelo_pib.joblib con manejo de errores.*

app_MLOps_reentrenamiento.py — línea 166

```python
# 4B. CONFIGURACIÓN DINÁMICA DE VARIABLES
# ============================================================

ruta_config = localizar_archivo(
    "config_modelo.json", ["salidas_tracker", "modelos", "."]
)
config = cargar_json(str(ruta_config)) if ruta_config else {}
nombres_entrenados = nombres_modelo_entrenado(modelo)

# Compatibilidad con el notebook anterior: mejor_experimento.json
# también contiene las variables elegidas en entrenamiento.
features_config = config.get("features")
if features_config is None and mejor_experimento is not None:
    features_config = mejor_experimento.get("features")
if features_config is None:
    features_config = nombres_entrenados

if not isinstance(features_config, list) or not features_config or not all(
    isinstance(v, str) and v.strip() for v in features_config
):
    st.error(
        "No se conoce el listado de variables del modelo. "
        "Suba config_modelo.json generado por Colab."
    )
    st.stop()

FEATURES = list(features_config)
TARGET = config.get("target", "crecimiento_pib")
if len(set(FEATURES)) != len(FEATURES):
    st.error("La configuración contiene variables duplicadas.")
    st.stop()

if nombres_entrenados is not None and nombres_entrenados != FEATURES:
    st.error(
        "Las variables de config_modelo.json no coinciden con las "
        "variables y el orden del modelo_pib.joblib. "
        "Vuelva a exportar ambos archivos desde la misma ejecución de Colab."
    )
    st.stop()

if nombres_entrenados is None:
    st.warning(
        "No fue posible contrastar los nombres de entrada guardados en el modelo. "
        "Verifique que la configuración y el .joblib sean de la misma ejecución."
    )

with st.expander(f"Variables del modelo activo ({len(FEATURES)})"):
    st.write(FEATURES)
```

Figura 23. *Configuración dinámica: variables leídas del JSON y contrastadas con el modelo.*

app_MLOps_reentrenamiento.py — línea 247

```python
with tab1:

    st.header("Predicción manual del PIB")

    st.write(
        "Ingrese los valores de los indicadores "
        "económicos utilizados por el modelo."
    )

    with st.form("formulario_manual"):
        valores = {}
        columnas = st.columns(2)
        for i, variable in enumerate(FEATURES):
            with columnas[i % 2]:
                valores[variable] = st.number_input(
                    variable.replace("_", " ").capitalize(),
                    value=0.0,
                    format="%.4f",
                    key=f"predictor_{variable}"
                )

        trimestre = st.text_input("Trimestre de estimación", value="2026Q4")
        escenario = st.selectbox(
            "Escenario", ["Base", "Optimista", "Pesimista", "Personalizado"]
        )
        calcular = st.form_submit_button("Generar predicción", type="primary")

    if calcular:
        datos_manual = pd.DataFrame([valores], columns=FEATURES)
        try:
            resultado = float(modelo.predict(datos_manual)[0])
            st.session_state.prediccion_manual = resultado
            st.session_state.registro_manual = {
                "trimestre": trimestre,
                "escenario": escenario,
                "PIB_estimado": resultado,
                "n_variables": len(FEATURES)
            }
        except Exception as e:
            st.session_state.prediccion_manual = None
            st.session_state.registro_manual = None
            st.error(f"Error al predecir: {e}")

    if st.session_state.prediccion_manual is not None:

        st.metric(
            "Crecimiento estimado del PIB",
            f"{st.session_state.prediccion_manual:.2f}%"
        )

        if st.button(
            "Registrar predicción manual",
            key="registrar_manual"
        ):

            st.session_state.historial.append(
                st.session_state.registro_manual.copy()
            )

            st.success("Predicción registrada.")
```

Figura 24. *Formulario de predicción manual generado a partir de FEATURES.*

app_MLOps_reentrenamiento.py — línea 1081

```
                historico_re = ordenar_y_validar(
                    pd.read_csv(archivo_historico), "Histórico"
                )
                nuevos_re = ordenar_y_validar(
                    pd.read_csv(archivo_nuevo), "Nuevos datos"
                )
                if historico_re["trimestre"].max() >= nuevos_re["trimestre"].min():
                    raise ValueError(
                        "Los nuevos trimestres deben ser posteriores a toda la "
                        "base histórica, sin solapamientos."
                    )
                if len(nuevos_re) < 4:
                    raise ValueError(
                        "Se requieren al menos cuatro trimestres nuevos con PIB "
                        "observado para entrenar y reservar una prueba posterior."
                    )
                # Reservar una ventana estrictamente posterior para comparar
                # ambos modelos sobre los MISMOS datos, sin entrenar con ellos.
                n_test = max(2, len(nuevos_re) // 4)
                nuevos_train = nuevos_re.iloc[:-n_test]
                holdout = nuevos_re.iloc[-n_test:]
                if len(nuevos_train) < 2:
                    raise ValueError("Insuficientes nuevos trimestres para entrenar")
                base_candidato = pd.concat(
                    [historico_re, nuevos_train], ignore_index=True
                )
                candidato = clone(modelo)
                candidato.fit(base_candidato[FEATURES], base_candidato[TARGET])
                y_real = holdout[TARGET].to_numpy()
                p_actual = modelo.predict(holdout[FEATURES])
                p_candidato = candidato.predict(holdout[FEATURES])
                rmse_actual = float(np.sqrt(mean_squared_error(y_real, p_actual)))
                rmse_candidato = float(np.sqrt(mean_squared_error(y_real, p_candidato)))
                mae_actual = float(mean_absolute_error(y_real, p_actual))
                mae_candidato = float(mean_absolute_error(y_real, p_candidato))
                aprobado = rmse_candidato < rmse_actual
                tabla_evaluacion = holdout[["trimestre", TARGET]].copy()
                tabla_evaluacion["prediccion_actual"] = p_actual
                tabla_evaluacion["prediccion_candidato"] = p_candidato

                reporte = {
                    "fecha_utc": datetime.now(timezone.utc).isoformat(),
                    "variables": FEATURES,
                    "target": TARGET,
                    "ultimo_trimestre_historico": str(historico_re["trimestre"].max()),
                    "ultimo_trimestre_nuevo": str(nuevos_re["trimestre"].max()),
                    "n_historico": len(historico_re),
                    "n_nuevos_entrenamiento": len(nuevos_train),
                    "n_holdout": len(holdout),
                    "inicio_holdout": str(holdout["trimestre"].iloc[0]),
                    "fin_holdout": str(holdout["trimestre"].iloc[-1]),
                    "rmse_actual": rmse_actual,
                    "rmse_candidato": rmse_candidato,
                    "mae_actual": mae_actual,
                    "mae_candidato": mae_candidato,
                    "supera_rmse_actual": aprobado,
                    "criterio": "RMSE del candidato menor, medido sobre el mismo holdout posterior",
                    "advertencia": "Comparación didáctica; con pocos trimestres la evidencia es limitada"
                }
                # Solo tras la comparación, ajustar un modelo descargable con
                # toda la historia disponible, incluyendo el holdout ya evaluado.
                final = clone(modelo)
                base_final = pd.concat([historico_re, nuevos_re], ignore_index=True)
                final.fit(base_final[FEATURES], base_final[TARGET])
                buffer = io.BytesIO()
                joblib.dump(final, buffer)
                st.session_state["resultado_reentrenamiento"] = {
                    "reporte": reporte,
                    "tabla": tabla_evaluacion,
                    "joblib": buffer.getvalue(),
                }
            except Exception as e:
                st.error(f"No fue posible completar el reentrenamiento: {e}")
```

Figura 25. *Reentrenamiento supervisado: el candidato se compara con el modelo actual en el mismo holdout.*

6.2 Ejecución

La aplicación se ejecutó localmente con el comando:

```
streamlit run app_MLOps_reentrenamiento.py
```

y queda disponible en `http://localhost:8501`. Las capturas de la sección 6.3 corresponden a esta ejecución local; la sección 7 muestra la misma aplicación desplegada en la nube.

6.3 Aplicación funcionando

Pantalla de inicio. Muestra el aviso de datos simulados, las cuatro variables del modelo activo (leídas de la configuración) y las cinco pestañas.

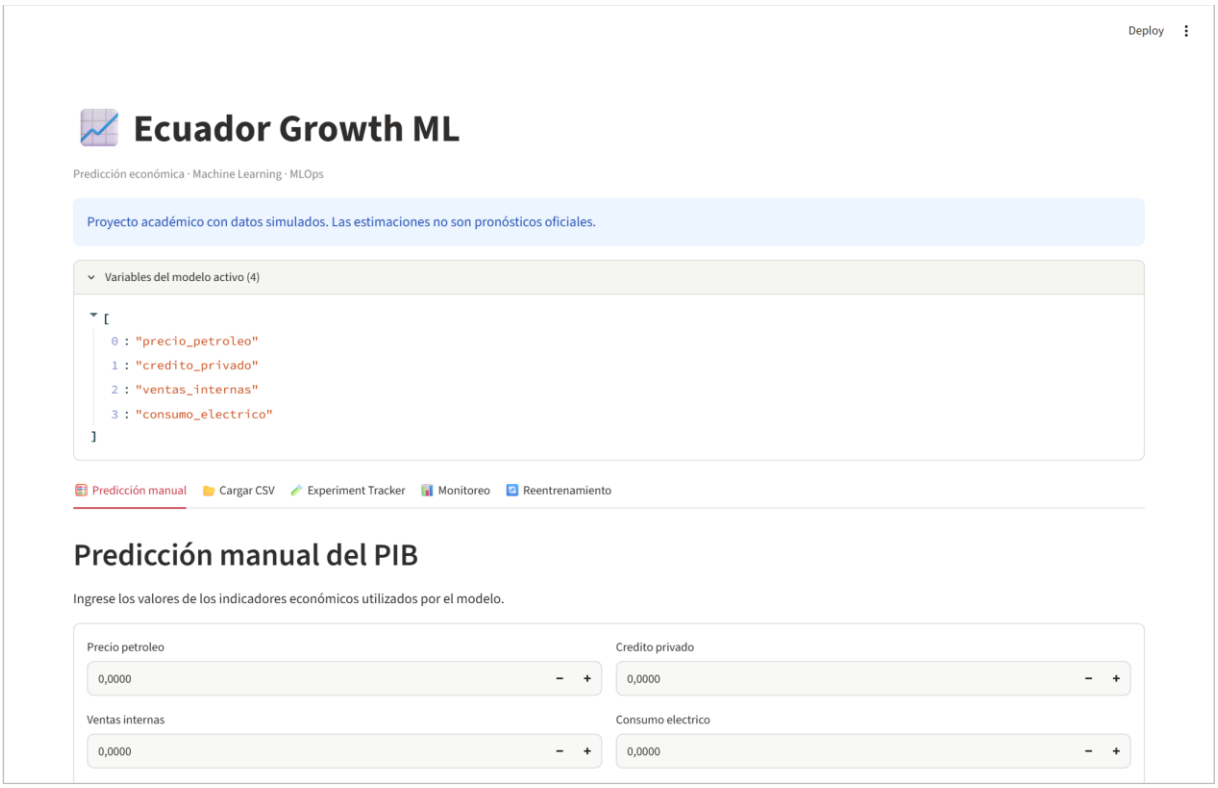


Figura 26. *Pantalla de inicio de Ecuador Growth ML.*

Predicción manual. Se ingresaron los indicadores de 2026Q4 del archivo `Indicadores_Actualizados_2026Q4_Ecuador_SIMULADOS.csv` (petróleo 88,111; crédito 2,654; ventas −1,202; electricidad 1,652). El modelo estima un **crecimiento del PIB de 1,57 %** y la predicción se registra en el historial.

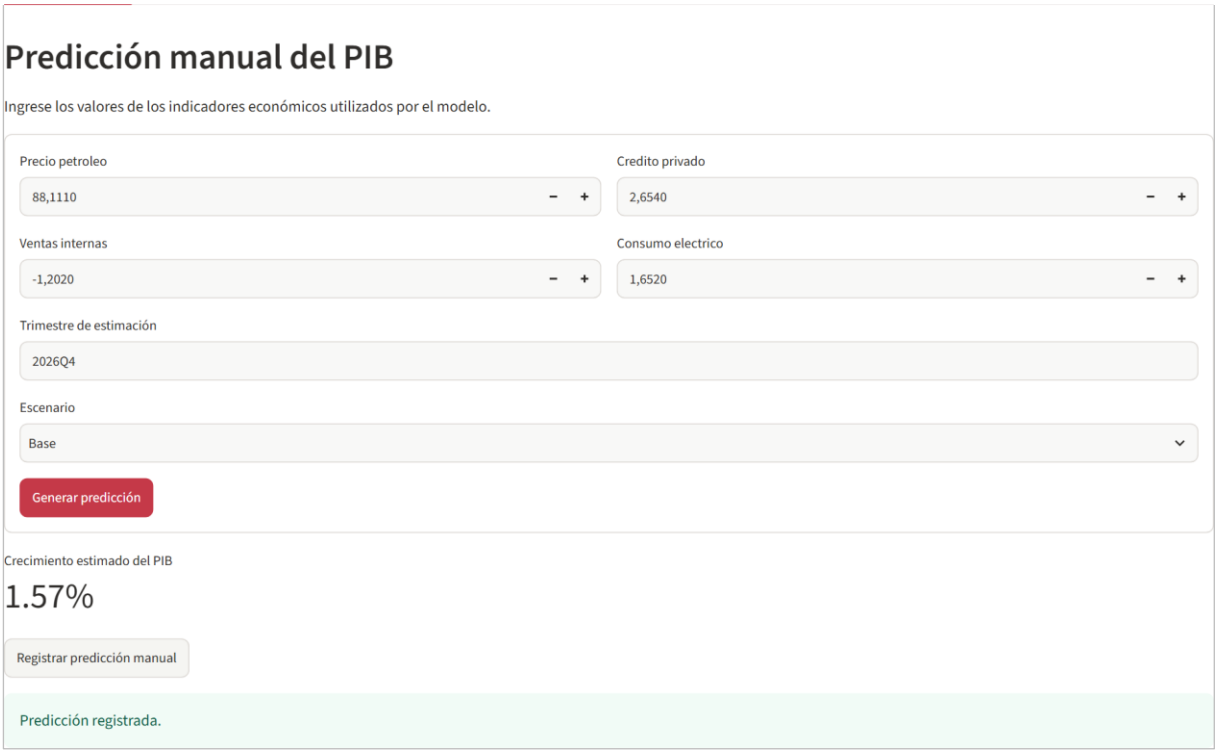


Figura 27. *Pestaña Predicción manual: nowcast de 2026Q4.*

Carga de CSV. Se cargó el archivo completo de indicadores de 2026Q4. La app toma solo las cuatro variables del modelo, genera la predicción (columna `PIB_estimado`, al final de la tabla) y la añade al historial, donde coincide con la predicción manual (1,5725).

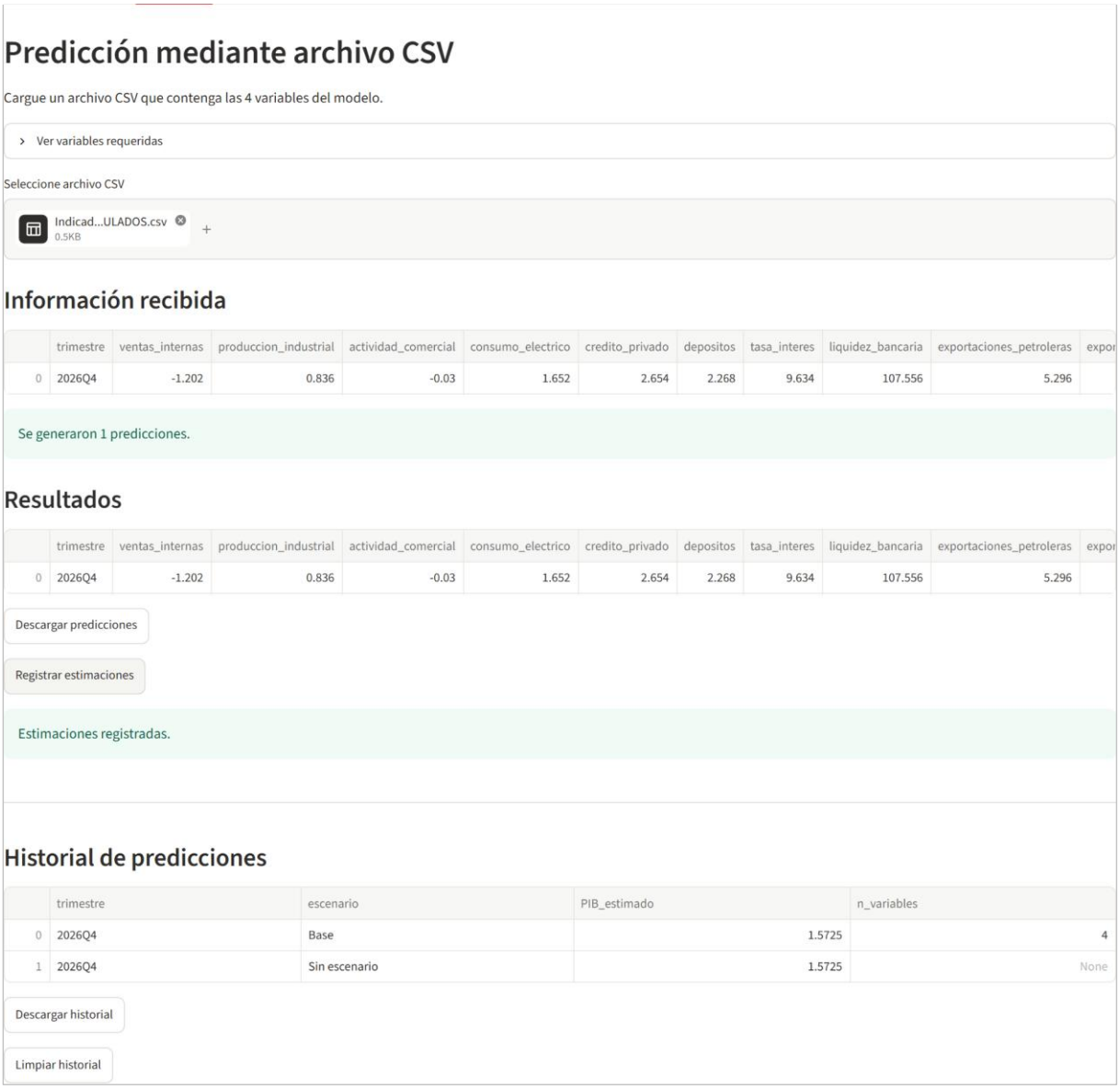


Figura 28. *Pestaña Cargar CSV: predicción por lotes e historial.*

Experiment tracker. Resume las 8 ejecuciones, identifica el ganador (RUN-003, RMSE CV 1,0192), permite comparar runs y muestra el error por ventana temporal y la configuración del mejor experimento.

ML Experiment Tracker

Seguimiento y comparación de experimentos de Machine Learning.

Ejecuciones	Mejor RMSE CV	Mejor MAE CV	Run ganador
8	1.0192	0.9236	RUN-003

Grilla de hiperparámetros

run_id	modelo	n_estimators	max_depth	min_samples_leaf	mae_cv	rmse_cv
RUN-003	Random Forest	100	7	2	0.9236	1.0192
RUN-007	Random Forest	200	7	2	0.9468	1.0449
RUN-001	Random Forest	100	4	2	0.9939	1.0987
RUN-005	Random Forest	200	4	2	1.0038	1.1124
RUN-004	Random Forest	100	7	4	1.0086	1.1167
RUN-008	Random Forest	200	7	4	1.0131	1.125
RUN-002	Random Forest	100	4	4	1.0289	1.1433
RUN-006	Random Forest	200	4	4	1.0312	1.1482

Descargar experimentos

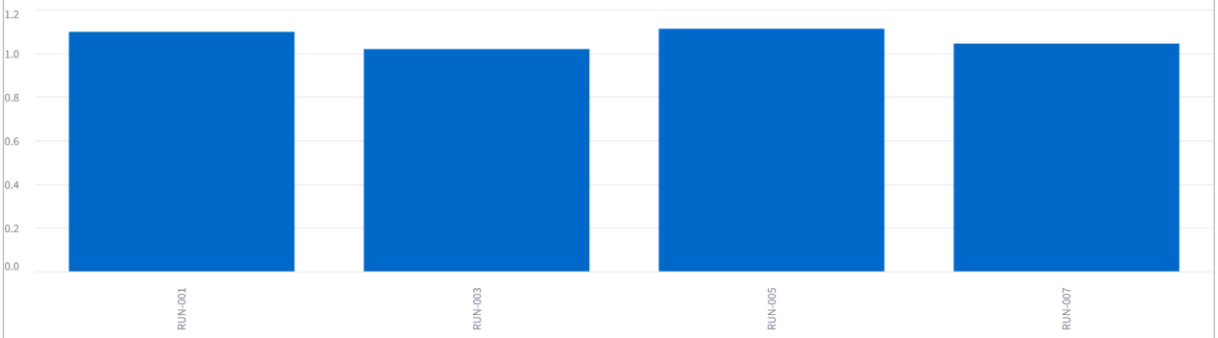
Comparación de modelos

Seleccione ejecuciones para comparar

RUN-003 × RUN-007 × RUN-001 × RUN-005 ×

Métrica de comparación

RMSE MAE



Menor error indica mejor desempeño predictivo.

Hiperparámetros comparados

run_id	modelo	n_estimators	max_depth	min_samples_leaf	mae_cv	rmse_cv
RUN-003	Random Forest	100	7	2	0.9236	1.0192
RUN-007	Random Forest	200	7	2	0.9468	1.0449
RUN-001	Random Forest	100	4	2	0.9939	1.0987
RUN-005	Random Forest	200	4	2	1.0038	1.1124

Detalle de ejecución

Seleccione un Run ID

RUN-003

Número de árboles

100

Profundidad máxima

7

Mínimo por hoja

2

MAE validación cruzada

0.9236

RMSE validación cruzada

1.0192

Validación cruzada temporal

run_id	fold	fin_entrenamiento	inicio_validacion	fin_validacion	mae	rmse
RUN-003	1	2020Q4	2021Q1	2021Q4	0.6919	0.7261
RUN-003	2	2021Q4	2022Q1	2022Q4	1.2249	1.3046
RUN-003	3	2022Q4	2023Q1	2023Q4	0.8212	0.8464
RUN-003	4	2023Q4	2024Q1	2024Q4	0.9566	1.1998



Mejor experimento

Experimento seleccionado: RUN-003


```
{
  "run_id" : "RUN-003"
  "parametros" : {
    "n_estimators" : 100
    "max_depth" : 7
    "min_samples_leaf" : 2
  }
```

```
"metricas_cv" : {
  "mae" : 0.9236408998345407
  "rmse" : 1.0192234407027712
}
"metricas_test" : {
  "run_id" : "RUN-003"
  "mae_test" : 0.6317158857055644
  "rmse_test" : 0.8301011652261744
  "r2_test" : 0.5718182607537095
  "n_test" : 7
  "inicio_test" : "2025Q1"
  "fin_test" : "2026Q3"
}
"features" : [
  0 : "precio_petroleo"
  1 : "credito_privado"
  2 : "ventas_internas"
  3 : "consumo_electrico"
]
}
```


Figura 29. *Pestaña Experiment Tracker. (3 partes)*

Monitoreo. Compara el PIB observado y el predicho en 2025Q1–2026Q3: MAE 0,6317 y RMSE 0,8301, con los gráficos de seguimiento y de errores, además del historial de predicciones de la sesión.

Monitoreo del modelo

Evaluación del comportamiento predictivo a partir de observaciones conocidas.

Predicciones del conjunto de prueba

	trimestre	pib_observado	pib_predicho	error
0	2025Q1	-2.055	-0.3622	-1.6928
1	2025Q2	-0.617	-0.1262	-0.4908
2	2025Q3	0.768	0.6465	0.1215
3	2025Q4	1.273	0.876	0.397
4	2026Q1	-0.659	0.3984	-1.0574
5	2026Q2	-0.8	-0.1471	-0.6529
6	2026Q3	1.84	1.8305	0.0095

Variables disponibles:


```
[
  0 : "trimestre"
  1 : "pib_observado"
  2 : "pib_predicho"
  3 : "error"
]
```

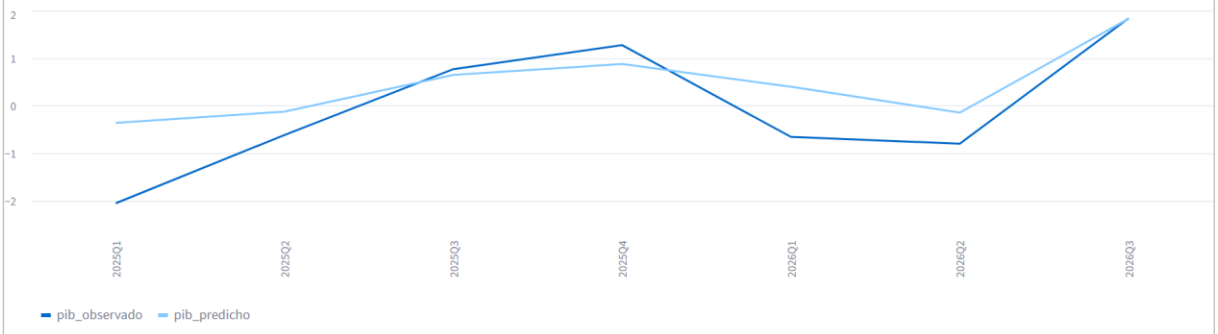

MAE de prueba

0.6317

RMSE de prueba

0.8301

Observado vs. predicho



Errores de predicción

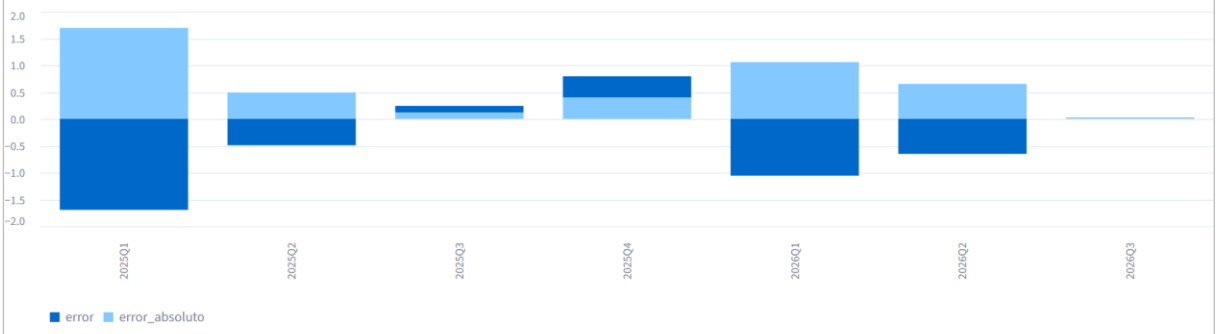




Figura 30. *Pestaña Monitoreo del modelo. (2 partes)*

Reentrenamiento supervisado. Se cargaron la base histórica y `nuevos_trimestres_PIB_simulados.csv` (8 trimestres, 2026Q4–2028Q3). La app entrena un candidato con los primeros 6 trimestres nuevos y compara ambos modelos en un holdout posterior (2028Q2–2028Q3). El candidato obtiene RMSE 0,2526 frente a 0,2502 del modelo actual, por lo que **no mejora** y la app recomienda conservar el modelo publicado. Es el comportamiento esperado de un control MLOps: no se reemplaza un modelo sin evidencia de mejora.

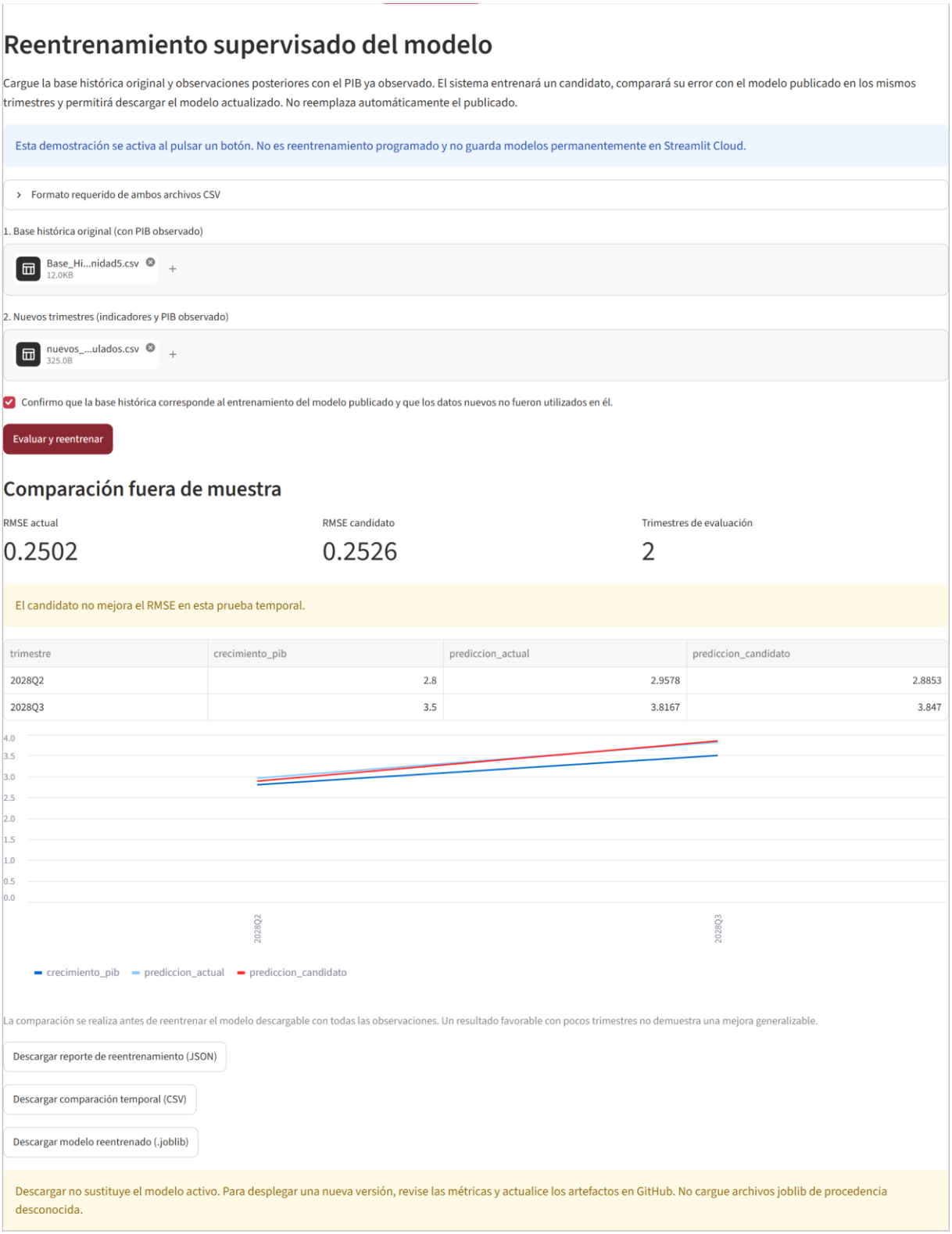


Figura 31. *Pestaña Reentrenamiento: comparación fuera de muestra.*

6.4 Nowcast del cuarto trimestre de 2026

Tabla 6. *Nowcast de 2026Q4 bajo dos escenarios de indicadores.*

Escenario (archivo)	Petróleo	Crédito	Ventas	Electricidad	PIB estimado
Base (`Indicadores_Actualizados_2026Q4…csv`)	88,111	2,654	−1,202	1,652	**1,57 %**
Alternativo (`Nuevos_Indicadores_PIB.csv`)	85,311	2,004	−2,102	1,202	**0,77 %**

Con indicadores más débiles en crédito, ventas, electricidad y petróleo, el nowcast cae de 1,57 % a 0,77 %. La dirección del cambio es coherente con la lógica económica de las variables elegidas.

7. Despliegue en Streamlit Community Cloud

La aplicación se publicó en Streamlit Community Cloud, conectada directamente al repositorio de GitHub. Cada cambio que se sube a la rama `main` actualiza la aplicación automáticamente, lo que cierra el ciclo MLOps: entrenamiento en Colab → artefactos versionados en GitHub → despliegue continuo.

Enlace público: https://appmlopsreentrenamientopy-aah9dtpk6lpswpjr3vp3vj.streamlit.app/

Tabla 7. *Configuración del despliegue.*

Parámetro	Valor
Repositorio	`VictorHugo1320/Ecuador_Growth_ML_APP`
Rama	`main`
Archivo principal	`app_MLOps_reentrenamiento.py`
Versión de Python	3.12 (la misma del entrenamiento del modelo)
Dependencias	`requirements.txt` (streamlit 1.67.0, scikit-learn 1.9.1, pandas 3.1.1, numpy 2.5.4, joblib 1.6.0)

Pasos del despliegue: (1) iniciar sesión en share.streamlit.io con la cuenta de GitHub; (2) Create app → seleccionar repositorio, rama y archivo principal; (3) en Advanced settings elegir Python 3.12; (4) Deploy. La primera instalación de dependencias tarda entre 2 y 5 minutos.

Verificación en la nube. Se repitieron en la aplicación publicada las mismas pruebas de la ejecución local, con resultados idénticos:

Tabla 8. *Pruebas de la aplicación desplegada en Streamlit Community Cloud.*

Pestaña	Prueba	Resultado en la nube
Predicción manual	Indicadores de 2026Q4	1,57 %
Cargar CSV	Indicadores_Actualizados_2026Q4…csv	1 predicción generada y registrada
Experiment Tracker	Lectura de salidas_tracker/	8 ejecuciones; ganador RUN-003 (RMSE CV 1,0192)
Monitoreo	Predicciones de prueba 2025Q1–2026Q3	MAE 0,6317; RMSE 0,8301
Reentrenamiento	Base histórica + 8 trimestres nuevos	RMSE actual 0,2502 vs. candidato 0,2526 (se conserva el modelo)

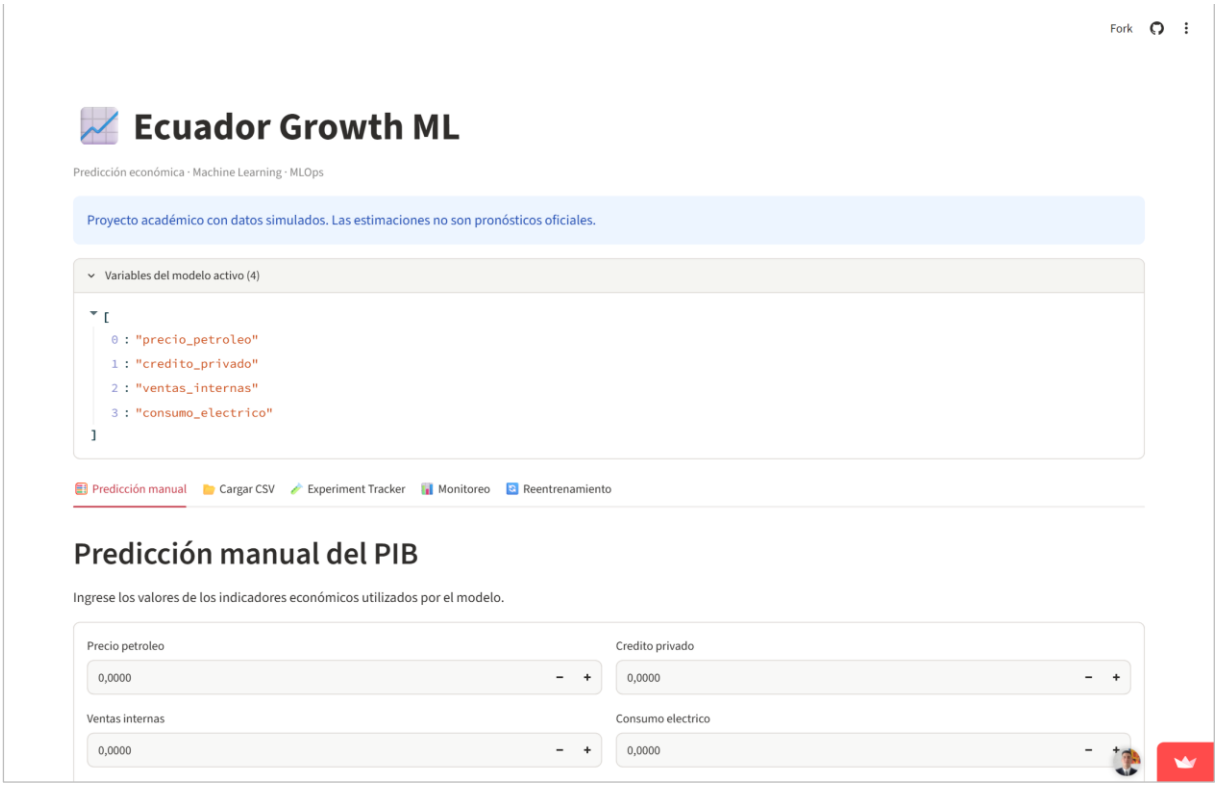


Figura 32. *Aplicación publicada en Streamlit Community Cloud: pantalla de inicio con las cinco pestañas.*

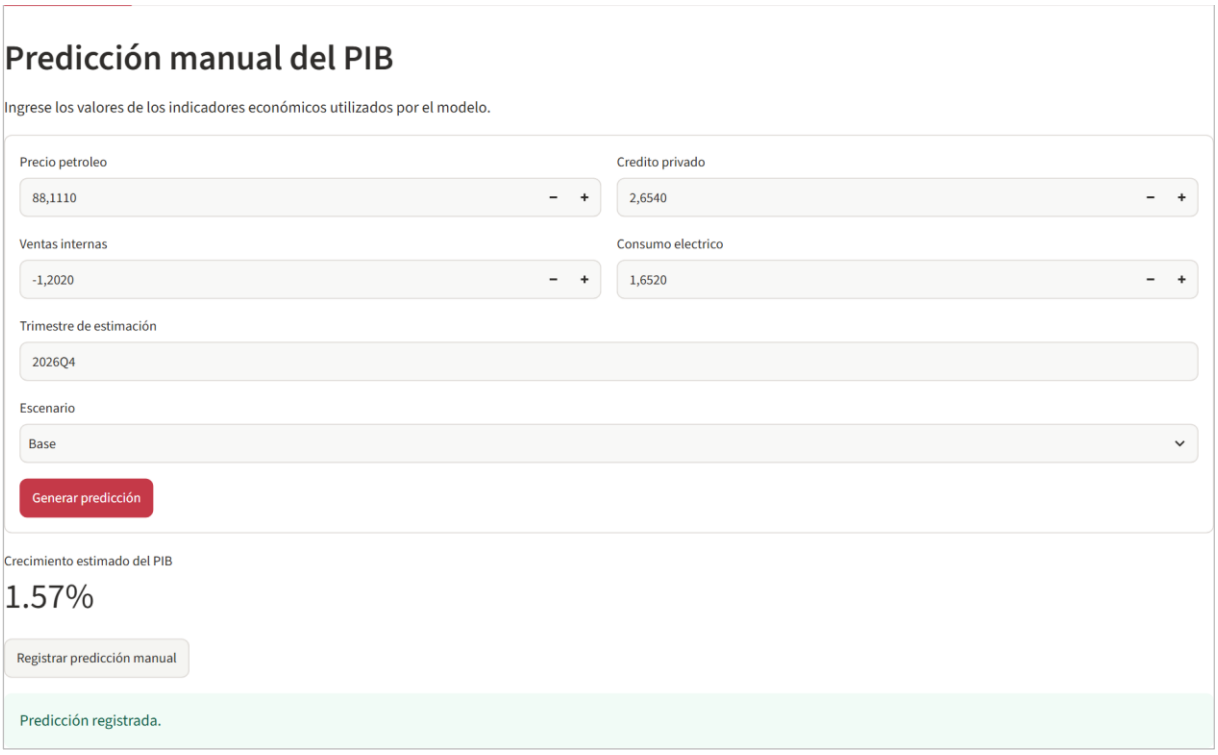


Figura 33. *Predicción manual en la nube: nowcast de 2026Q4 = 1,57 %.*

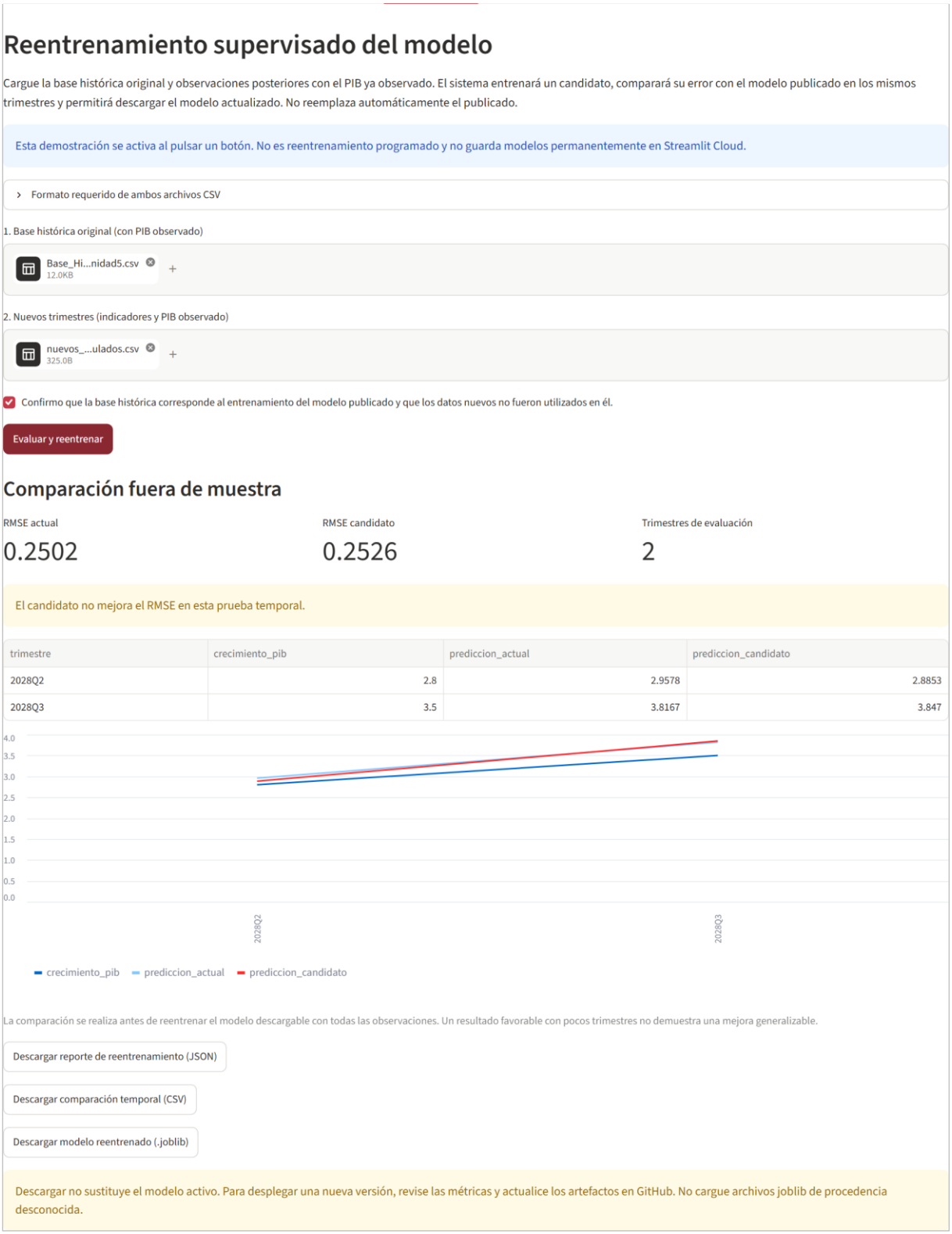


Figura 34. *Reentrenamiento supervisado ejecutado en la nube.*

8. Ajustes realizados

- **Rutas portables en el notebook:** lee los CSV desde `/content/` en Colab o desde la carpeta local; la descarga con `google.colab.files` solo se ejecuta dentro de Colab.

- **Sección de justificación de variables** añadida al notebook (correlaciones, importancia y comparación de conjuntos por validación cruzada).
- **Corrección en la pestaña Monitoreo:** la app buscaba las columnas `PIB_real` y `prediccion`, pero el notebook exporta `pib_observado` y `pib_predicho`, por lo que no calculaba las métricas. Se añadieron esos nombres a la búsqueda en `app.py` y `app_MLOps_reentrenamiento.py`.
- **Archivo requirements.txt** con versiones fijas (streamlit 1.67.0, scikit-learn 1.9.1, pandas 3.1.1, numpy 2.5.4, joblib 1.6.0), necesario para que Streamlit Cloud cargue el `.joblib` con la misma versión de scikit-learn con que se entrenó.

9. Conclusiones

- Un conjunto pequeño y económicamente justificado de cuatro indicadores (petróleo, crédito privado, ventas internas y consumo eléctrico) pronostica mejor que usar las 20 candidatas o las de mayor correlación: RMSE de validación de 1,081 frente a 1,253 y 1,337.
- El crédito privado es el predictor más valioso para anticipar el ciclo en una economía dolarizada; el precio del petróleo y el consumo eléctrico aportan información complementaria del sector externo y de la actividad real.
- El modelo ganador del experiment tracker (RUN-003) alcanza en prueba un RMSE de 0,83 p.p. y un R^2 de 0,57, y estima un crecimiento del PIB de 1,57 % para 2026Q4 en el escenario base.
- La aplicación está publicada en Streamlit Community Cloud con despliegue continuo desde GitHub y reproduce en la nube los mismos resultados que la ejecución local.
- La aplicación implementa el ciclo MLOps completo: configuración dinámica, predicción, trazabilidad de experimentos, monitoreo y reentrenamiento con control de calidad antes de reemplazar el modelo.
- Limitaciones: los datos son simulados, la muestra de prueba es corta (7 trimestres) y el Random Forest suaviza los valores extremos, como se observó en 2025Q1.

Anexo. Entorno de ejecución y archivos entregados

Tabla 9. *Versiones utilizadas en la ejecución.*

Componente	Versión
Python	3.12.3
scikit-learn	1.9.1
pandas	3.1.1
numpy	2.5.4
joblib	1.6.0
Streamlit	1.67.0

Tabla 10. *Archivos de la entrega.*

Archivo	Contenido
`Modelo_Nowcast_Dinamico_Ejecutado.ipynb`	Notebook con todas las celdas ejecutadas y la justificación de variables
`app_MLOps_reentrenamiento.py` / `app.py`	Aplicación Streamlit (con y sin pestaña de reentrenamiento)
`modelo_pib.joblib`, `salidas_tracker/`	Artefactos del modelo y del experiment tracker
`requirements.txt`	Dependencias para el despliegue
Este documento (Word y PDF)	Documentación con capturas de código y de la aplicación